APPLICATION PLATFORM STANDARDS — NX-OPS-K8S-NP-006

Kubernetes Best Practices — Non-Production Environments

Nexlify Technologies | Cluster: nexlify-nonprod-eks | Region ap-south-1 | Dev & Staging namespaces | Rev 2.2

1. Design Philosophy for Non-Production Clusters

Non-production clusters deliberately prioritise cost efficiency and developer velocity over high availability and strict service-level objectives. The financial target is to keep the monthly EC2 spend attributable to non-production workloads under twenty-five percent of the corresponding production compute cost while still providing an environment realistic enough for integration and load testing. All worker nodes are provisioned exclusively from Spot capacity through Karpenter; on-demand fallback is disabled. Stateful services that require persistence use PersistentVolumeClaims backed by gp3 EBS volumes. For pure development namespaces, data loss caused by Spot interruption is accepted; staging namespaces may enable volume snapshots for overnight recovery points. Horizontal Pod Autoscaler objects are not created for application workloads—replica counts remain fixed and intentionally small. PodDisruptionBudgets are omitted because voluntary disruptions are frequent and expected. Resource requests are set to the minimum viable values observed during local testing, with no additional over-provisioning buffer. The same AZ preference used elsewhere in the platform (ap-south-1a primary) applies so that PVC-backed pods stay co-located with their volumes and cross-AZ data-transfer charges stay minimal.

2. One-Hundred-Percent Spot and Node Placement Constraints

Every NodePool that serves the nexlify-nonprod-eks cluster declares capacity-type: spot and does not list on-demand. Instance families are diversified across m6i, m6a, m7i, r6i and c6i so that a capacity shortage in any single family does not block scheduling. Topology spread and affinity rules prefer Availability Zone ap-south-1a for any pod that mounts a PersistentVolumeClaim, because EBS volumes are zone-scoped. Developers are instructed to avoid unnecessary nodeSelector or affinity statements; the combination of the default scheduler and Karpenter will place the majority of workloads correctly. When a Spot interruption warning is received, Karpenter drains the node and launches a replacement; pods that own PVCs automatically reschedule into the same Availability Zone so that the volume can be re-attached without cross-AZ data movement. Node expiration is set to thirty days so that AMI and security updates are picked up regularly without manual intervention.

3. Container Image Size and Probe Timing Guidelines

Container images must be kept as small as practical. Distroless or Alpine-based runtimes are preferred; multi-stage builds are mandatory for any compiled language. The size target for an application container image is under 150 megabytes. Probe timings are deliberately short so that developers receive rapid feedback when a pod fails to start: initialDelaySeconds is set to 5, periodSeconds to 5, timeoutSeconds to 2, and failureThreshold to 3 for both liveness and readiness probes. Startup probes may increase the failureThreshold for services that are known to be slow to initialise. Readiness gates and custom pre-stop hooks are not required in non-production unless the service is being used for load-generation tests that intentionally mimic production traffic patterns. Image digests are preferred over mutable tags even in non-prod so that the same image can later be promoted without rebuild.

4. Features Explicitly Disabled in Non-Production

Horizontal Pod Autoscaler resources are not installed for application namespaces; replica counts stay static. The cluster-proportional autoscaler may still adjust CoreDNS replica count. PodDisruptionBudget resources are omitted so that Karpenter can freely consolidate and recycle nodes. Resource requests are set equal to the expected steady-state usage; limits may be higher, but the difference should not exceed twenty percent—there is no intentional over-provisioning buffer. PriorityClass objects other than the cluster default are reserved for platform components that use system-cluster-critical. NetworkPolicy objects are recommended for namespaces that handle credentials or personally identifiable information, yet they are not enforced cluster-wide in non-production. These omissions keep the cluster simple to operate and cheap to run while still allowing realistic functional testing.

5. Namespace Isolation and Resource Quotas

Each product squad receives a dedicated Kubernetes namespace protected by a ResourceQuota that caps aggregate CPU and memory consumption. LimitRange objects supply default request and limit values for containers that omit them. When a squad exhausts its quota, developers open a ticket against the Platform Engineering queue for a temporary increase; permanent increases require a capacity-planning review. Namespace labels include the owning squad, cost centre and environment so that charge-back reports and network-policy selectors can be constructed automatically. Admission controllers reject any Deployment that requests more than the remaining quota. Soft limits are communicated via documentation and Slack reminders rather than hard blocks, preserving developer velocity.

6. Operational Notes and Common Pitfalls

Teams sometimes attempt to copy production manifests (HPA, PDB, high resource requests) into non-prod namespaces. Those objects are harmless if present but defeat the cost and simplicity goals of the environment; Platform Engineering periodically audits and removes them. Another frequent issue is placing PVC-backed pods without zone affinity, which can cause Pending pods after Spot interruption when the volume remains in ap-south-1a while the replacement node lands in ap-south-1b. The NodePool topology constraints and the recommended affinity patterns above prevent that failure mode. For questions about non-prod capacity or quota increases, contact the Application Platform squad via Slack channel #k8s-nonprod.

Owner: Application Platform | Slack: #k8s-nonprod | Revision 2.2 | Last updated 2026-08-31 | Related: NX-OPS-K8S-PR-007 (prod standards), NX-OPS-KARP-005